30 Production Incidents Every Senior Java Engineer Should Recognize

Symptom -> Root Cause -> Fix -> Prevention

Real failure patterns across concurrency, databases, memory/JVM, distributed systems, and Spring configuration — each with the root cause and the fix, not just the symptom.

Compiled by Syed Minhaj (Sam) | Senior Java Full Stack Developer / Tech Lead
Toronto, Ontario, Canada

If you've shipped production Java systems for a few years, you've likely lived through several of these already — even if you never had a name for the root cause.

Contents

1. Concurrency & Threading

Silent transaction rollback failure

SYMPTOM

Inventory goes negative after bulk orders; no error in logs.

ROOT CAUSE

@Transactional method called internally (this.method()) bypasses the Spring AOP proxy, so no transaction boundary is ever created for that call.

FIX

Move the transactional method to a separate injected bean, or self-inject the proxy via @Lazy as a stopgap.

PREVENTION

Static analysis rule or code review checklist item: flag any @Transactional method invoked from within the same class.

Thread pool exhaustion cascades into full outage

SYMPTOM

One slow downstream API causes the entire service to stop responding, not just that one endpoint.

ROOT CAUSE

A shared, undersized thread pool with no timeout on the downstream call let every incoming request pile up waiting on the same blocked resource.

FIX

Isolate thread pools per downstream dependency (bulkhead pattern) and set explicit connect/read timeouts on every outbound call.

PREVENTION

No HTTP client call ships without an explicit timeout, enforced in code review and via a shared HTTP client wrapper.

Request data leaking across unrelated users

SYMPTOM

User A occasionally sees User B's data in logs or, worse, in a response.

ROOT CAUSE

ThreadLocal used for request-scoped context (user ID, tenant ID) was never cleared in a finally block; pooled threads carried stale values into the next request.

FIX

Wrap ThreadLocal set/get in a try/finally that always calls remove(), or use a request-scoped Spring bean instead.

PREVENTION

Any ThreadLocal usage requires a paired .remove() in the same code review, no exceptions.

Counter values drift under load

SYMPTOM

A request counter or rate-limit counter reports numbers lower than actual traffic.

ROOT CAUSE

volatile int used for a compound read-modify-write (counter++), which is not atomic despite the visibility guarantee volatile provides.

FIX

Replace with AtomicInteger/AtomicLong, or use a proper synchronized block around the read-modify-write.

PREVENTION

Treat any counter or accumulator shared across threads as a hard requirement for an atomic type, not a plain primitive.

Deploys cause a spike in duplicate-processed events

SYMPTOM

Every rolling deploy correlates with a burst of duplicate order confirmations or duplicate emails.

ROOT CAUSE

Kafka consumer group rebalancing during deploy reprocesses messages that were handled but not yet offset-committed.

FIX

Make the consumer's business logic idempotent (unique event ID + DB uniqueness constraint) rather than relying on exactly-once delivery alone.

PREVENTION

Idempotency is a required design review item for any new Kafka consumer, not an optional hardening step.

---

2. Database & Persistence

API response time degrades linearly with data growth

SYMPTOM

A list endpoint that was fast at launch is now consistently slow, worst on high page numbers.

ROOT CAUSE

OFFSET-based pagination forces the database to scan and discard every prior row before returning the requested page.

FIX

Switch to keyset (cursor-based) pagination using an indexed column (WHERE id > :lastId ORDER BY id LIMIT n).

PREVENTION

Default all new paginated endpoints to cursor-based pagination; require justification for OFFSET.

Dashboard load time balloons after adding a related-entity field

SYMPTOM

A single UI change (showing customer name next to each order) multiplies query count by order count.

ROOT CAUSE

Classic N+1: lazy-loaded association accessed in a loop, firing one query per row instead of one query total.

FIX

Use JOIN FETCH or @EntityGraph for the specific query path that needs the association, or configure batch fetching.

PREVENTION

Enable SQL statement logging in staging and review query counts per request during code review for any new list endpoint.

Two services deadlock every few hours under load

SYMPTOM

Random transaction timeouts and deadlock exceptions in logs, seemingly unrelated to any single deploy.

ROOT CAUSE

Two code paths acquire row-level locks (SELECT ... FOR UPDATE) on the same two rows but in opposite order.

FIX

Enforce a consistent lock acquisition order (e.g., always ascending primary key) across every code path touching those rows.

PREVENTION

Document and enforce lock ordering conventions for any multi-row pessimistic locking pattern.

Users report a purchase 'succeeded twice' or 'disappeared'

SYMPTOM

Rare, hard-to-reproduce reports of a booking or purchase being duplicated or silently overwritten.

ROOT CAUSE

A check-then-act pattern (check availability, then decrement) run under the database's default READ COMMITTED isolation level, allowing a race condition between the check and the act.

FIX

Use SELECT ... FOR UPDATE for pessimistic locking, or optimistic locking via a @Version field with explicit conflict handling.

PREVENTION

Any check-then-act business logic on shared state is flagged in design review as needing explicit concurrency control.

Application serves data that contradicts what's in the database

SYMPTOM

A batch job updates a table directly, but the application keeps returning old values for hours.

ROOT CAUSE

Second-level Hibernate cache was enabled for the entity, and the batch job's direct SQL write bypassed Hibernate entirely, never invalidating the cache.

FIX

Explicitly evict the relevant cache region after any out-of-band write, or route the batch job through the same persistence layer.

PREVENTION

Any process that writes to a cached entity's table outside the application must be documented and paired with a cache-eviction step.

A previously fast query suddenly takes seconds after a data migration

SYMPTOM

No code change, but query latency jumps after a large data import or partition growth.

ROOT CAUSE

Stale table statistics after bulk insert caused the query planner to choose a sequential scan instead of an available index.

FIX

Run ANALYZE (or equivalent) on the affected table after any large bulk write, and verify via EXPLAIN ANALYZE.

PREVENTION

Bulk-load runbooks include a mandatory statistics refresh step as the final task.

---

3. Memory & JVM

Service OOMs every few days, always after steady growth

SYMPTOM

Heap usage climbs continuously under constant load and never drops after a full GC, eventually crashing.

ROOT CAUSE

A static Map used as an unbounded cache with no eviction policy, holding entries indefinitely.

FIX

Replace with a bounded cache (Caffeine with maximumSize or expireAfterWrite) or an explicit eviction strategy.

PREVENTION

Any static or long-lived collection used as a cache requires an explicit size or TTL bound before merge.

Native memory usage climbs independently of heap usage

SYMPTOM

Heap looks healthy in monitoring, but the container gets OOM-killed by the OS.

ROOT CAUSE

Metaspace grew unbounded due to heavy dynamic proxy generation (CGLIB) from a large number of Spring beans, with no MaxMetaspaceSize cap set.

FIX

Set an explicit -XX:MaxMetaspaceSize and monitor Metaspace usage separately from heap in your dashboards.

PREVENTION

JVM startup flags for every service include explicit Metaspace and heap bounds, reviewed as part of the deployment template.

Cold-start latency is unacceptable in a serverless/container environment

SYMPTOM

First requests after scale-up or deploy are dramatically slower than steady-state requests.

ROOT CAUSE

JIT compilation hasn't warmed up yet; the interpreter is executing bytecode for methods that haven't been profiled as 'hot' and compiled to native code.

FIX

Use tiered compilation tuning, consider AOT/CDS (Class Data Sharing) or a startup-optimized runtime for latency-sensitive cold-start scenarios.

PREVENTION

Load/warm-up requests as part of the deployment health check before routing real traffic to a new instance.

GC pause spikes correlate with a specific downstream traffic pattern

SYMPTOM

p99 latency spikes to seconds during specific windows, with no corresponding code deploy.

ROOT CAUSE

Upstream data shape change (e.g., significantly larger payloads) increased allocation rate, triggering more frequent and longer GC pauses under the existing collector configuration.

FIX

Profile allocation rate, and evaluate a low-latency collector (ZGC/Shenandoah) if p99 pause time is the binding constraint.

PREVENTION

GC pause metrics are part of standard service dashboards, not something checked only when an incident occurs.

Memory usage is high but stable, yet performance still degrades over time

SYMPTOM

Heap plateaus at a high but non-crashing level; throughput slowly drops rather than the service crashing outright.

ROOT CAUSE

Large objects retained longer than necessary (e.g., a full result set collected into a List and held in scope across an entire request), increasing GC scan time without technically leaking.

FIX

Narrow variable scope, stream/process data instead of materializing full collections where possible, and profile with a heap dump to distinguish retention from leaks.

PREVENTION

Treat 'stable but high' memory as worth investigating, not just 'not an OOM so it's fine.'

---

4. Distributed Systems & Microservices

One dependency outage takes down three unrelated services

SYMPTOM

A single third-party API degradation causes cascading failures across multiple internal services that call it directly or indirectly.

ROOT CAUSE

No circuit breaker; every caller kept retrying and timing out against the failing dependency, exhausting their own thread pools in the process.

FIX

Introduce circuit breakers (Resilience4j) with fail-fast behavior and a cooldown period before retrying the dependency.

PREVENTION

Any new external dependency call requires a circuit breaker and timeout as part of the integration checklist.

A payment is charged twice after a client-side retry

SYMPTOM

Customer complaints about duplicate charges specifically correlated with slow network conditions on the client side.

ROOT CAUSE

No idempotency key on the payment endpoint; a client timeout-and-retry resulted in two independent, both-successful executions of the same logical operation.

FIX

Require a client-supplied idempotency key, checked and stored atomically before processing, returning the cached result on retry.

PREVENTION

Any endpoint that mutates state and could reasonably be retried by a client must support idempotency keys before launch.

A multi-step business process gets stuck in a permanently inconsistent state

SYMPTOM

An order shows as 'paid' but inventory was never decremented, with no automatic recovery.

ROOT CAUSE

A multi-service transaction had no compensating action defined for a partial failure — there was no saga, just a hopeful sequence of calls.

FIX

Implement the Saga pattern with explicit compensating transactions for each step, orchestrated centrally for visibility.

PREVENTION

Any operation spanning more than one service's data store requires an explicit failure/compensation design before implementation.

A poison-pill message halts an entire event pipeline

SYMPTOM

Message processing for an entire partition stops; consumer lag climbs indefinitely on that partition only.

ROOT CAUSE

A single malformed message causes an unhandled exception on every retry attempt, and the consumer never advances past it.

FIX

Add a dead-letter topic with DeadLetterPublishingRecoverer after N retries, committing the offset to unblock the partition.

PREVENTION

Every Kafka consumer ships with dead-letter handling from day one, not added reactively after the first incident.

Health checks say everything is fine, but users see errors

SYMPTOM

Load balancer shows all instances healthy while real user requests are failing or timing out intermittently.

ROOT CAUSE

Health checks only verify the instance is up, not that its downstream dependencies (DB, cache, another service) are actually reachable and performant.

FIX

Add fine-grained request-level resilience (mesh-level or library-level circuit breaking) in addition to coarse instance-level health checks.

PREVENTION

Distinguish liveness checks (is the process up) from readiness checks (can it actually serve traffic) in every service's health endpoint design.

---

5. Spring, Configuration & Security

A security check silently never runs

SYMPTOM

Access control that appeared correct in code review doesn't actually block unauthorized access in production.

ROOT CAUSE

@PreAuthorize on a method called internally within the same class bypasses the AOP proxy, same root cause as the @Transactional issue.

FIX

Restructure so the secured method is called through the proxy (separate bean) or via the injected self-proxy.

PREVENTION

Any annotation relying on Spring AOP (@Transactional, @PreAuthorize, @Async, @Cacheable, @Retryable) is checked for self-invocation in review.

A bean silently isn't wired the way a config change intended

SYMPTOM

A new @ConditionalOnProperty-gated bean doesn't activate even though the property looks correctly set.

ROOT CAUSE

A typo or environment-specific property override (profile-specific YAML, environment variable precedence) meant the condition never evaluated true.

FIX

Run with the auto-configuration debug report enabled to see exactly why the conditional evaluated as it did.

PREVENTION

Configuration changes affecting bean creation get a staging verification step using the auto-config report before production deploy.

An audit trail has gaps exactly where things went wrong

SYMPTOM

The one operation you most need an audit record for is missing it after a rollback.

ROOT CAUSE

Audit logging used REQUIRED transaction propagation inside the larger business transaction, so it rolled back along with the failed business logic.

FIX

Use REQUIRES_NEW propagation for audit/logging writes that must persist independently of the outer transaction's outcome.

PREVENTION

Any audit or compliance-relevant write is explicitly reviewed for correct propagation semantics, not left at the default.

Read replicas are provisioned but barely reduce primary DB load

SYMPTOM

Read replica utilization stays low while the primary remains the bottleneck, despite read-heavy traffic.

ROOT CAUSE

Service methods that only read data were never marked readOnly = true, so read-replica routing never kicked in and everything hit the primary.

FIX

Explicitly mark genuinely read-only service methods with @Transactional(readOnly = true) and verify replica routing is configured to use that signal.

PREVENTION

readOnly = true is a required checklist item for any new query-only service method.

A conflict error becomes a confusing 500 instead of a clean response

SYMPTOM

Users see a generic server error when they hit a legitimate optimistic-locking conflict from concurrent edits.

ROOT CAUSE

OptimisticLockException was never caught; it propagated as an unhandled exception instead of a deliberate 409 response.

FIX

Catch OptimisticLockException explicitly at the service or controller-advice layer and translate it into a 409 Conflict with a clear message.

PREVENTION

Any entity using @Version has a corresponding, tested exception-handling path, not just the annotation.

Every senior engineer's judgment is built from incidents like these. Recognizing the pattern before it happens is what the title is actually for.

Compiled by Syed Minhaj (Sam) — Senior Java Full Stack Developer / Tech Lead, Toronto, ON. Connect on LinkedIn for more posts on Java internals, Spring, and system design.